
DOCUMENT TECHNIQUE — CotiPay

TCC Hack & Defend 2026 | Track : FinTech Équipe : Commit & Pray

PAGE 1 — Stack Technologique

React.js

Rôle : Construction de l'interface utilisateur **Pourquoi** : Permet de découper l'interface en composants réutilisables. Sa gestion de l'état avec useState et useEffect facilite l'affichage en temps réel des données sans rechargement de page.

Tailwind CSS

Rôle : Stylisation de l'interface **Pourquoi** : Permet de styliser directement dans le JSX sans fichier CSS séparé. Gain de temps significatif pour une équipe travaillant sous contrainte de temps.

Vite

Rôle : Outil de build et serveur de développement **Pourquoi** : Démarrage quasi-instantané du serveur de développement, rechargement à chaud automatique à chaque modification.

Node.js + Express.js

Rôle : Serveur backend et API REST **Pourquoi** : Même langage (JavaScript) côté frontend et backend, ce qui réduit la charge cognitive pour l'équipe. Express est minimaliste et rapide à mettre en place pour des API REST.

PostgreSQL

Rôle : Base de données relationnelle **Pourquoi** : Les données financières nécessitent des relations solides entre les tables (membres, cotisations, micro-finances). PostgreSQL garantit l'intégrité des données avec les contraintes FOREIGN KEY et CHECK.

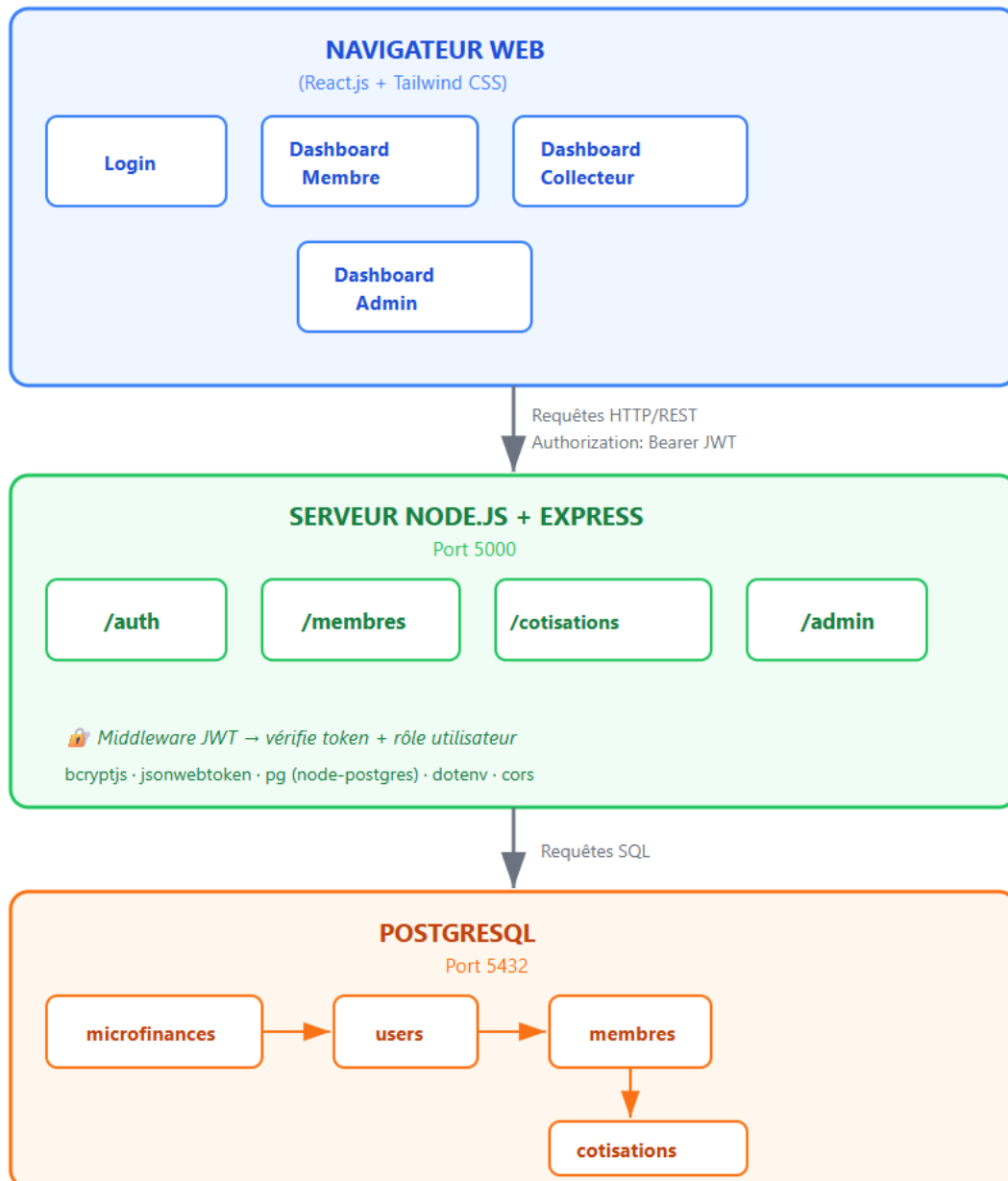
JWT (JSON Web Token)

Rôle : Authentification et autorisation **Pourquoi** : Permet de gérer les 3 rôles distincts (admin, collecteur, membre) de manière stateless. Le token est stocké côté client et envoyé à chaque requête protégée.

bcryptjs

Rôle : Hachage des mots de passe **Pourquoi :** Sécurise les mots de passe en les hachant avant stockage en base. Aucun mot de passe n'est stocké en clair.

2 — Architecture de la Solution



Rôle de chaque composante :

- **React Frontend** : Affiche les données et envoie les actions utilisateur vers l'API
- **API REST Node.js** : Reçoit les requêtes, vérifie les tokens JWT, exécute la logique métier
- **Middleware JWT** : Intercepte chaque requête protégée, vérifie le token et le rôle
- **PostgreSQL** : Stocke toutes les données de manière relationnelle et sécurisée

3 — Fonctionnalités Implémentées

CE QUI FONCTIONNE COMPLÈTEMENT

- Inscription et connexion sécurisée des 3 rôles (admin, collecteur, membre)
- Authentification JWT avec redirection automatique selon le rôle
- Hachage bcrypt des mots de passe
- Dashboard Membre : solde en temps réel + historique des cotisations
- Dashboard Collecteur : liste des membres assignés + validation de cotisation en 1 clic
- Mise à jour automatique du solde du membre après validation
- Dashboard Admin : statistiques globales (total membres, cotisations du jour, total collecté)
- Tableau des cotisations du jour avec nom du membre, montant, statut
- Déconnexion sécurisée (suppression du token localStorage)
- Protection des routes par rôle (un collecteur ne peut pas accéder aux routes admin)

PARTIELLEMENT IMPLÉMENTÉ

- Calendrier de cotisation (l'historique est affiché mais pas sous forme de calendrier visuel mensuel)
- Gestion multi micro-finances (la structure SaaS est en place mais l'interface de création de nouvelles micro-finances n'est pas encore développée)

NON IMPLÉMENTÉ (manque de temps)

- Intégration mobile money réelle (T-Money, Flooz) — simulée par validation manuelle du collecteur
- Demande de retrait en ligne pour le membre
- Notifications push quand le collecteur valide
- Application mobile
- Export PDF des rapports mensuels pour l'admin

4 — Difficultés Rencontrées

1. Connexion réseau entre les PC de l'équipe

Problème : Le frontend sur un PC ne pouvait pas atteindre le backend sur un autre PC à cause des restrictions réseau et du pare-feu Windows. **Solution :** Chaque développeur a installé PostgreSQL localement sur son PC et partagé le même fichier .env avec les bonnes configurations.

2. Incompatibilité entre les noms de champs

Problème : La route de login utilisait password comme nom de champ mais le frontend envoyait mot_de_passe, causant une erreur 401 systématique. **Solution :** Uniformisation du nom du champ à mot_de_passe dans toutes les routes et le frontend.

3. Requêtes SQL sur les mauvaises tables

Problème : Plusieurs routes cherchaient m.nom et m.prenom dans la table membres alors que ces colonnes sont dans la table users. **Solution :** Ajout de jointures JOIN users u ON m.user_id = u.id dans toutes les requêtes concernées.

4. Gestion des branches Git en équipe

Problème : Première expérience de travail collaboratif sur Git, plusieurs conflits de merge rencontrés. **Solution :** Chaque membre a travaillé sur sa propre branche, les merges ont été faits via Pull Requests sur GitHub.

5. Hash bcrypt incompatible

Problème : Les mots de passe insérés en base avec un hash générique ne correspondaient pas à la version de bcryptjs installée. **Solution :** Génération des hash directement depuis Node.js avec bcryptjs.hash() et mise à jour en base via psql.

5 — Perspectives

Court terme

- Intégration des APIs mobiles money T-Money et Flooz pour les paiements réels
- Ajout du calendrier visuel mensuel dans le dashboard membre
- Interface de demande de retrait pour le membre

Moyen terme

- Application mobile React Native
- Système de notifications push (SMS ou WhatsApp)
- Tableau de bord analytique avancé pour les admins (graphiques, exports)

- Gestion multi micro-finances depuis un super-admin CotiPay

Long terme

- Partenariats officiels avec les micro-finances agréées du Togo
- Modèle d'abonnement : 10 000 FCFA/mois par micro-finance
- Extension à d'autres pays d'Afrique de l'Ouest

Document rédigé par l'équipe Commit & Pray — TCC Hack & Defend 2026

#TCCHackDefend2026 | CotiPay — Digitalisons la tontine togolaise
